Introduction to Haskell Programming

User-defined data types

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

User-defined data types

- The `data` keyword is used to define new types

User-defined data types

- The `data` keyword is used to define new types
- Enumerated data types:

```
data Bool   = False | True
data Day    = Sun | Mon | Tue | Wed | Thu | Fri | Sat
```

User-defined data types

- The `data` keyword is used to define new types
- Enumerated data types:

```
data Bool   = False | True
data Day    = Sun | Mon | Tue | Wed | Thu | Fri | Sat
```

- Data types with values that have parameters:

```
data Shape  = Circle Double | Square Double | Rectangle Double Double

shapes :: [Shape]
shapes      = [Circle 3.0, Square 4.0, Rectangle 3.0 4.0]
```

Functions on data types

- Functions can be defined using pattern matching

```
weekend :: Day → Bool
weekend Sat = True
weekend Sun = True
weekend _   = False

area :: Shape → Double
area (Circle r)         = pi * r * r
area (Square s)         = s * s
area (Rectangle l b)    = l * b
```

Functions on data types

- What about the following function?

```
weekend2 :: Day → Bool
weekend2 d  = d == Sat || d == Sun
```

Functions on data types

- What about the following function?

```
weekend2 :: Day → Bool
weekend2 d  = d == Sat || d == Sun
```

- Error!

```
-- No instance for (Eq Day) arising from a use of '=='
```

Functions on data types

- What about this function?

```
nextday :: Day → Day
nextday Sun = Mon
nextday Mon = Tue
...
nextday Sat = Sun
```

Functions on data types

- What about this function?

```
nextday :: Day → Day
nextday Sun = Mon
nextday Mon = Tue
...
nextday Sat = Sun
```

- Invoke `nextday Fri` in `ghci`

Functions on data types

- What about this function?

```
nextday :: Day → Day
nextday Sun = Mon
nextday Mon = Tue
...
nextday Sat = Sun
```

- Invoke `nextday Fri` in `ghci`
- Error again!

```
-- No instance for (Show Day) arising from a use of 'print'
```

Adding data type instances

- To check equality of two values of a data type a ...

Adding data type instances

- To check equality of two values of a data type `a` ...
- there must be an `Eq a` instance

Adding data type instances

- To check equality of two values of a data type `a` ...
- there must be an `Eq a` instance
- We use `deriving` to create such instances:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving Eq
```

Adding data type instances

- To check equality of two values of a data type `a` ...
- there must be an `Eq a` instance
- We use `deriving` to create such instances:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving Eq
```

- Default behaviour – `Sun == Sun`, `Tue /= Fri`, ...

Adding data type instances

- To check equality of two values of a data type `a` ...
- there must be an `Eq a` instance
- We use `deriving` to create such instances:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving Eq
```

- Default behaviour – `Sun == Sun`, `Tue /= Fri`, ...
- Now `weekday2` compiles without error

Adding data type instances

- To make `nextday` work, we must make an instance for **`Show`** `Day`

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show)
```

Adding data type instances

- To make `nextday` work, we must make an instance for **`Show`** `Day`

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show)
```

- **`show`** provides a default text representation that can be printed on screen

Adding data type instances

- To make `nextday` work, we must make an instance for **`Show`** `Day`

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show)
```

- **`show`** provides a default text representation that can be printed on screen
- **`show`** `Wed = "Wed"`

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`
- Can also create an `Enum Day` instance

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord, Enum)
```

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`
- Can also create an `Enum Day` instance

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord, Enum)
```

- Can now write these expressions:

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`
- Can also create an `Enum Day` instance

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord, Enum)
```

- Can now write these expressions:
  - `succ Tue`

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`
- Can also create an `Enum Day` instance

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord, Enum)
```

- Can now write these expressions:
  - `succ Tue`
  - `pred Fri`

Adding data type instances

- Can also create an `Ord Day` instance:

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord)
```

- Default behaviour: `Sun < Mon < Tue < Wed < Thu < Fri < Sat`
- Can also create an `Enum Day` instance

```
data Day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
    deriving (Eq, Show, Ord, Enum)
```

- Can now write these expressions:
  - `succ Tue`
  - `pred Fri`
  - `[Sun .. Thu]`

Adding data type instances

- Instances for `Shape`:

```
data Shape = Circle Double | Square Double | Rectangle Double Double
    deriving (Eq, Ord, Show)
```

Adding data type instances

- Instances for `Shape`:

```
data Shape = Circle Double | Square Double | Rectangle Double Double
    deriving (Eq, Ord, Show)
```

- Default behaviours:

```
show (Circle 5.0) == "Circle 5.0"
Square 4.0 == Square 4.0
Square 4.0 /= Square 3.0
Circle 5.0 /= Rectangle 3.0 4.0
[Square 2.0, Circle 3.0, Square 22.0] < [Square 2.0, Square 0.005]
```

Adding custom instances

- One can define custom instances with non-default behaviour

Adding custom instances

- One can define custom instances with non-default behaviour
- The keyword `instance` is used

Adding custom instances

- One can define custom instances with non-default behaviour
- The keyword **`instance`** is used
- **Note:** We cannot use **`deriving`** and **`instance`** w.r.t. the same type and type class!

```
instance Ord Shape where
    (⩽) :: Shape → Shape → Bool
    shape1 ⩽ shape2 = area shape1 ⩽ area shape2
```

Adding custom instances

- One can define custom instances with non-default behaviour
- The keyword **instance** is used
- **Note:** We cannot use **deriving** and **instance** w.r.t. the same type and type class!

```
instance Ord Shape where
    (⩽) :: Shape → Shape → Bool
    shape1 ⩽ shape2 = area shape1 ⩽ area shape2
```

- The $\leqslant$ on the left is **being defined**

Adding custom instances

- One can define custom instances with non-default behaviour
- The keyword `instance` is used
- **Note:** We cannot use `deriving` and `instance` w.r.t. the same type and type class!

```
instance Ord Shape where
    (⩽) :: Shape → Shape → Bool
    shape1 ⩽ shape2 = area shape1 ⩽ area shape2
```

- The $\leqslant$ on the left is **being defined**
- The $\leqslant$ on the right is **being used** and is of type `Double → Double → Bool`

Adding custom instances

- A custom **Show** instance of Shape

```
instance Show Shape where
    show :: Shape → String
    show (Circle r)      = "Circle with radius " ++ show r
    show (Square s)      = "Square with side " ++ show s
    show (Rectangle l b) = "Rectangle with length " ++ show l
                                  ++ " breadth " ++ show b
```

Adding custom instances

- A custom **Show** instance of Shape

```
instance Show Shape where
    show :: Shape → String
    show (Circle r)      = "Circle with radius " ++ show r
    show (Square s)      = "Square with side " ++ show s
    show (Rectangle l b) = "Rectangle with length " ++ show l
                                  ++ " breadth " ++ show b
```

- The **show** on the right is of type **Double → String**

Constructors

- `Square`, `Circle`, `Sun`, `Mon`, ...are **constructors**

Constructors

- `Square`, `Circle`, `Sun`, `Mon`, ...are **constructors**
- They are just functions, but start with an uppercase letter

```
Sun :: Day
Rectangle :: Double → Double → Shape
Circle :: Double → Shape
```

Constructors

- `Square`, `Circle`, `Sun`, `Mon`, ...are **constructors**
- They are just functions, but start with an uppercase letter

```
Sun :: Day
Rectangle :: Double → Double → Shape
Circle :: Double → Shape
```

- They can be used just like any other function

```
Circle 5.0 :: Shape
map Circle :: [Double] → [Shape]
map Circle [2.0, 3.0] = [Circle 2.0, Circle 3.0]
```

Records

- Data types with a **single constructor**

Records

- Data types with a **single constructor**
- **Convention**: The single constructor has the same name as the type

```
data Person = Person String Int Double String
    deriving (Eq, Show)
gal = Person "Ashvini" 21 5.9 "ashvini@me.com"
```

Records

- Data types with a **single constructor**
- **Convention**: The single constructor has the same name as the type

```
data Person = Person String Int Double String
    deriving (Eq, Show)
gal = Person "Ashvini" 21 5.9 "ashvini@me.com"
```

- The four parameters are supposed to stand for **name**, **age**, **height** and **email id**

Records

- Data types with a **single constructor**
- **Convention**: The single constructor has the same name as the type

```
data Person = Person String Int Double String
    deriving (Eq, Show)
gal = Person "Ashvini" 21 5.9 "ashvini@me.com"
```

- The four parameters are supposed to stand for **name**, **age**, **height** and **email id**
- How do we extract the height of `gal`?

Records

- Data types with a **single constructor**
- **Convention**: The single constructor has the same name as the type

```
data Person = Person String Int Double String
    deriving (Eq, Show)
gal = Person "Ashvini" 21 5.9 "ashvini@me.com"
```

- The four parameters are supposed to stand for **name**, **age**, **height** and **email id**
- How do we extract the height of `gal`?
- We need **destructors**

Destructors

```
name :: Person → String
name (Person n _ _ _) = n

age :: Person → Int
age (Person _ a _ _) = a

height :: Person → Double
height (Person _ _ h _) = h

email :: Person → String
email (Person _ _ _ e) = e
```

Record syntax

- We can name the fields:

```
data Person = Person {name :: String, age :: Int,
                      height :: Double , email :: String
                      } deriving Show
```

Record syntax

- We can name the fields:

```
data Person = Person {name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

- We can name fields while creating values of type `Person`

```
gal = Person {name = "Ashvini", height = 5.9,
    email = "ashvini@me.com", age = 21}
```

Record syntax

- We can name the fields:

```
data Person = Person {name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

- We can name fields while creating values of type `Person`

```
gal = Person {name = "Ashvini", height = 5.9,
    email = "ashvini@me.com", age = 21}
```

- Order of fields not important

Record syntax

- We can name the fields:

```
data Person = Person {name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

- We can name fields while creating values of type `Person`

```
gal = Person {name = "Ashvini", height = 5.9,
    email = "ashvini@me.com", age = 21}
```

- Order of fields not important
- The following also works, but fields have to be in order!

```
gal = Person "Ashvini" 21 5.9 "ashvini@me.com"
```

Record syntax

- We can name the fields:

```
data Person = Person {
            name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

Record syntax

- We can name the fields:

```
data Person = Person {
            name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

- The field names are actually functions

Record syntax

- We can name the fields:

```
data Person = Person {
            name :: String, age :: Int,
            height :: Double , email :: String
            } deriving Show
```

- The field names are actually functions
- Automatically defined for us when we use record syntax

```
name :: Person → String
age :: Person → Int
height :: Person → Double
email :: Person → String
```

Complex numbers

- We can define complex numbers as a data type

```
data Complex = Complex Double Double deriving Eq
```

Complex numbers

- We can define complex numbers as a data type

```
data Complex = Complex Double Double deriving Eq
```

- Can define functions by pattern matching as usual

```
absVal :: Complex → Double
absVal (Complex x y) = sqrt (x^2 + y^2)
```

Complex numbers

- We can define complex numbers as a data type

```
data Complex = Complex Double Double deriving Eq
```

- Can define functions by pattern matching as usual

```
absVal :: Complex → Double
absVal (Complex x y) = sqrt (x^2 + y^2)
```

- Can define a custom `Ord` and `Show` instances

```
instance Ord Complex where
    (⩽) :: Complex → Complex → Bool
    cmplx1 ⩽ cmplx2 = absVal cmplx1 ⩽ absVal cmplx2
instance Show Complex where
    show (Complex x y) = show x ++ " + i " ++ show y
```

Complex numbers

- How do we extract the real and imaginary parts of a complex number?

```
re, im :: Complex → Double
re (Complex x _) = x
im (Complex _ y) = y
```

Complex numbers

- How do we extract the real and imaginary parts of a complex number?

```
re, im :: Complex → Double
re (Complex x _) = x
im (Complex _ y) = y
```

- Can use record syntax also and avoid defining the destructors explicitly

```
data Complex = Complex {re :: Double, im :: Double}
    deriving (Eq, Show)

absVal :: Complex → Double
absVal cmplx = sqrt ((re cmplx)^2 + (im cmplx)^2)
```

User-defined polymorphic types

- We can also define our own polymorphic types!

User-defined polymorphic types

- We can also define our own polymorphic types!
- Our own version of lists whose elements are of type a

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Ord)
```

User-defined polymorphic types

- We can also define our own polymorphic types!
- Our own version of lists whose elements are of type `a`

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Ord)
```

- `List` is not a type

User-defined polymorphic types

- We can also define our own polymorphic types!
- Our own version of lists whose elements are of type `a`

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Ord)
```

- `List` is not a type
- It is a **type constructor**!

User-defined polymorphic types

- We can also define our own polymorphic types!
- Our own version of lists whose elements are of type `a`

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Ord)
```

- `List` is not a type
- It is a **type constructor**!
- For any type `a`, `List a` is a type

User-defined polymorphic types

- We can also define our own polymorphic types!
- Our own version of lists whose elements are of type `a`

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Ord)
```

- `List` is not a type
- It is a **type constructor**!
- For any type `a`, `List a` is a type
- `Nil` and `Cons` are **value constructors** or **data constructors**

```
Nil :: List a
Cons :: a → List a → List a
```

User-defined polymorphic types

- Two values of type `List a` can be compared (for equality or order) ...

User-defined polymorphic types

- Two values of type `List a` can be compared (for equality or order) ...
  - ...provided the type `a` itself is an instance of `Eq` and `Ord`

User-defined polymorphic types

- Two values of type `List a` can be compared (for equality or order) ...
  - ...provided the type `a` itself is an instance of `Eq` and `Ord`
  - This is automatically enforced by `deriving`

User-defined polymorphic types

- Two values of type `List a` can be compared (for equality or order) ...
  - ...provided the type `a` itself is an instance of `Eq` and `Ord`
  - This is automatically enforced by `deriving`
- We can create a custom `Show` instance, provided `a` itself is an instance of `Show`

```
instance Show a ⇒ Show (List a) where
    show l              = "[" ++ go l ++ "]" where
        go Nil          = ""
        go (Cons x Nil) = show x
        go (Cons x l)   = show x ++ ", " ++ go l
```